Cryptography Class Notes

These notes are for the first few classes of a course taught from Hoffstein Pipher and Silverman's 'An Introduction to Mathematical Cryptography'. which is herein refereed to as 'the text'. They draw largely from the text but reorder it a bit: bringing Sections 1.6 and 2.5 forward. I number the sections to try to reflect the this.

This is an upper level undergraduate first class in Cryptography.

1 An Introduction to Cryptography

This class is about *Cryptography*, a subfield of *Coding Theory*. For context, Coding Theory is the study of *codes*. The main types of codes are:

i). *Secret codes* - used to transfer information so that only the intended recipient can read it.

ii). *Compression codes* - used to transfer information efficiently.

iii). *Error correcting codes* - used to correctly transfer information that might get corrupted in transfer.

'Cryptography' is usually taken to be the study of secret codes, and this is what we will study. 'Coding theory' can refer to everything, or to the study of the last two types of codes.

1.1 Intro to the Intro: Simple Substitution Cipher

Example 1.1. Say Alice wanted to send a message to Bob, but didn't want Eve to be able to read it.

Tomorrow at Lunch

She could replace every letter with the letter to the left of it on the keyboard, and instead send

Ypzpttpe sy Aimvj

Eve can't read her message, but Bob knows he has to replace every letter with the letter to the left of it on the keyboard, and can convert it back to the original message.

What Alice and Bob have used is called a *simple substitution cipher*. It is an example of a scheme called a *cryptosystem*. The elements of a cryptosystem are, informally:

- The message 'tomorrow at lunch' is the *plaintext*.
- The message 'YPZPTTPE SY AIMVJ' is the *ciphertext*.
- The algorithm for converting between ciphertext and the plaintext is called the *key* or *cipher*.
- The process of converting the plaintext to the ciphertext is called *encryption*.
- The process of converting the ciphertext to the plaintext using the key is called *decryption*.
- The process of converting the ciphertext to the plaintext without the key is called *cryptanalysis*, or *cracking* the cipher.

Simple substitution ciphers are and obvious cryptosystem and have historically been very useful. But they can be cracked quite easily. Especially for long messages.

1.1.1 Cracking a Simple Substitution Cipher

The most obvious approach is to try every possible substition of the letters and see which gives some proper English words. But 26! is a pretty big number and this would be tricky even with computers. This is the approach, but one has to be a bit more clever. Just a bit though, this approach is still quite obvious. It appears in Poe's 'The Gold Bug'.

The letter 'e' occurs most frequently in the English language, accounting for about .13 of all letters. Then comes 't' at about .10 and then 'a' at about .08. Given a long enough ciphertext, the basic idea is to count the occurences of each symbol, replace the most frequent with an 'e', the next most frequent with a 't', etc.

Sure this isn't going to work out exactly all the time. Indeed it usually won't. But the most common letters are 'e, t, a, o, n'. Try putting these randomly into the 8 or so most frequent ciphertext symbols, you have a much more managable number of combinations. You can check these for agreement with words in the english language: there should be no two letter words 'ta' or 'ea'. Much better.

One can also use the frequency of two letter combinations. The most common two letter combinations, or digraphs, in English are 'th' 'he' and 'in'. Choosing 't' and 'e' in the first step, if there is some ciphertext letter X such that tX and Xe are frequent, you can be pretty sure that this is an h.

And if you have a three letter work 'PYQ' occuring several times you can be pretty sure this 'the' or 'and'.

With a bit of math, but more contextual reasoning, it is not hard to solve a simple substition cipher. But it is hard enough to be fun. That is why there are Word Jumbles. http://www.wordjumble.com/ Give them to your Dad and tell him they fight off Alzheimier's.

Problem 1.1. Reading Section 1.1.1 of the text if necessary, do Problem 1.4 (a) from the text

1.1.2 Private vs. Public key Cryptosystems

In the simple substitution cipher, it is required that Alice and Bob both know the key. Such cyptosystems are called *private key* cryptosystems, or *symmetric* cryptosystems.

Contrastingly, in a *public key* cryptosystem, or *asymmetric* cryptosystem, there are two keys, one for encryption, and one for decryption. In such a cryptosystem, Bob keeps the decryption key secret, but makes the encryption key public. Under this scheme, anyone, including Alice, can send Bob a message, which only Bob can read.

There are advantage to both private key and public key systems.

As we saw, private key cryptosystems can be cracked, but this is not really one of their shortcomings. There are many different private key cryptosystems, and it is not hard to come up with one that is impossible to crack without the key.

Problem 1.2. Come up with at private key cryptosystem that is impossible to crypoanalyse, and explain why it is impossible.

But this suggests one of the shortcomings. What if Bob loses the key? Wiki up the story of the Enigma code. Those Germans were using a private key cryptosystem called the Enigma - a fancy sort of shift cipher with a progressive shift. They were sneaking around in their submarines wreaking German havoc. Nobody could crack their code. Not even the guy from Rubicon. But then the Americans or the British, or probably the Canadians, caught one of their submarines and got the key. End of story, Germans.

If you want a code that many people can use, and any one person loses the private key, the code is compromised.

In a public key cryptosystem, anyone can have the encryption key. Alice doesn't care if Bob number 17 gives it up to Eve, she even makes it public, Eve still can't decode other peoples ciphertexts.

Another shortcoming of the private key cryptosystem is that Alice and Bob must first meet to share the key, and this is not always practical.

While there are uncrackable private key cryptosystems, public key cryptosystems, by necessity, can be cracked. What makes one good, is that it is difficult to crack. In this course, except in the first few classed, will look exclusively at public key cryptosystems, and the mathematics behind them.

1.6 A more rigourous introduction

This is mostly Section 1.6 of the text. We make our definitions more rigourous, and lay out the rules of the game- those assumptions that we will be making to allow us to get to the math of the subject.

1.6.1 Encryption Schemes

A *cipher* or *encryption scheme* is a tuple $(\mathcal{K}, \mathcal{M}, \mathcal{C}, e, d)$ where $\mathcal{M}$ is a space of *plaintexts* or messages or, $\mathcal{C}$ is a space of *ciphertexts*, $\mathcal{K}$ is a space of *keys*, e is function

$$e : \mathcal{K} \times \mathcal{M} \to \mathcal{C},$$

called *encryption*, and d is a function

$$d : \mathcal{K} \times \mathcal{C} \to \mathcal{K}$$

called *decryption*, such that $d(k, e(k, m)) = m$ for any key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$.

This is more notation that is useful for our purposes. Usually we assume that the encryption scheme is understood and for a fixed key k write the encryption and decryption functions

$$e_k : \mathcal{M} \to \mathcal{C} : m \mapsto e(k, m)$$

and

$$d_k : \mathcal{C} \to \mathcal{M} : c \mapsto d(k, c).$$

For public key cryptosystems, we said there is an encryption key k and a decryption key k_d. However one can always compute k_d from k, so we really have only one key. The differce between public and private key systems is that for public key systems, k_d is hard to compute from k.

Informally, we will often call e_k the encryption key, and d_k the decryption key, thinking of the key as the function, rather than the number it is based on. When we analyse an encryption scheme, we assume that the enemy, Eve, knows the encryption scheme, but not the (decryption) key.

Clearly, to have $d_k(e_k(m)) = m$, e must be injective and d must be surjective. Other properties that we want of a cipher are

P1) $e_k(m)$ is easy to compute for $k \in \mathcal{K}$ and $m \in \mathcal{M}$.

P2) $d_k(c)$ is easy to compute given k and c.

P3) Given many $c_1, \ldots, c_n \in \mathcal{C}$, it is difficult to compute any of the $d_k(c_i)$ without having k.

P4) Given several pairs $(c_1, d_k(c_1)), \ldots, (c_n, d_k(c_n))$, it is difficult to compute $d_k(c)$ for some $c \notin \{c_1, \ldots, c_n\}$.

Having a pair $(c_i, d_k(c_i))$ is equivalent to having $(e_k(m), m)$ for some $m \in \mathcal{M}$, because we know $e_k(m)$ decodes to m.

Properties P3 and P4 essentially say that cryptoanalysis is hard.

1.6.2 Plaintexts and Ciphertexts as number strings

In substitution cipher example we saw earlier, the key was a little hard to describe: move once key to the right on the keyboard. Here is more how we will do a substitution cipher. It is called a simple shift substitution cipher.

Associating the ith letter of the alphabet with the number $i-1$, (so $a = 0$ and $b = 1$ and, ..., $z = 25$,) we can write our message " Tomorrow at Lunch" as

$$19\,14\,12\,14\,17\,17\,14\,22 \qquad 00\,19 \qquad 11\,20\,13\,02\,07.$$

Now to encrypt it we pick a secret key $k \in \{0, \ldots, 25\}$ and add it to each letter, modulo 26. Taking key $k = 9$ we encode to

$$03\,23\,21\,23\,01\,01\,23\,06 \qquad 09\,03 \qquad 20\,04\,22\,11\,16.$$

Now perhaps we would use the same letter to number key, and send this as

Dxvxbbxgjduewlq,

but let's just let that be overhead, and always consider our plaintext and ciphertext as numbers.

In this example, our encryption scheme is the 'simple shift cipher'. This tells us that the $\mathcal{M}, \mathcal{C}$, and $\mathcal{K}$ are all $\mathbb{Z}_{26}$ the integers modulo 26. Our actual message consisted of a string of 15 plaintexts in $\mathcal{M}$ which we encrypt and decrypt individually, and the spaces between them.

Having $k = 9$ our encryption key is

$$e_k : m \mapsto m + 9 \text{ modulo } 26$$

and our decryption key is

$$d_k : m \mapsto m - 9 \text{ modulo } 26.$$

This is a symmetric cryptosystem, as d_k is easy to deduce from k or e_k.

We will usually encode messages as binary strings. So with that previous example could encode

$$19\ 14\ 12\ 14\ 17\ 17\ 14\ 22 \qquad 00\ 19 \qquad 11\ 20\ 13\ 02\ 07.$$

into length 5 binary strings as:

$$10011\ 01110\ 01100\ 01110\ 10001\ 10001\ 01110\ 10110 \qquad 00000\ \ldots$$

We didn't apply e to the whole message at once, but to each letter at a time. This is weak, in that is does not satisfy property P4. This is because, $\mathcal{M}$ is too small. Once we know the encoding of each letter we can decode the message. Generally we will string several letters together to get an element of $\mathcal{M}$. For example, we would encode the above message as length 20 binary strings:

$$10011011100110001110100 01\ 10001011101011011111100000\ \ldots$$

where we have encoded the empty space as 11111. This is also considered a part of the encoding, and not of the encryption scheme.

The length B of the strings in $\mathcal{M}$ is called the *blocksize* of the cipher. Often we have to pad to final plaintext block so that the message fits into it.

Generally $\mathcal{M}, \mathcal{C}$, and $\mathcal{K}$ are all regular length binary strings. We use B_m, B_c and B_k to refer to their respective blocklengths.

It is generally accepted that with present computing power, a cipher is safe against an exhaustive search attack if the blocklength is at least 80.

More Examples of Symmetric Ciphers

Let p be a large prime ($p \approx 2^{160}$), and $\mathcal{M} = \mathcal{C} = \mathcal{K}$ be the integers modulo p. We will study these a bit later, but for now, if you are unfamiliar with them, just pretend that you are not.

Let encryption

$$e_k(m) = k \cdot m.$$

Then

$$d_k(m) = k^{-1} \cdot m$$

is decryption.

Given k, it takes about $2\log_2(p) = 320$ steps calculations to find k^{-1}. A computer can do this in seconds, so this is easy. (We don't have to find k^{-1} for each decryption, it is part of the key that we only have to generate once. So it isn't one of the properties P1 - P4 we mentioned. But it is desirable– it is inconvenient if it takes too long to generate a key.)

Encryption and decryption are easy, so we have properties P1 and P2. Since e_k is surjective, whatever k is, some message encodes to the ciphertext c_i, so

having a couple ciphertexts doesn't help us decrypt anything. So we have property P3. However we don't have property P4. As soon as you have some pair $(c, m = d_k(c))$, you know that $c = km$ so you can compute

$$c \cdot m^{-1} = km \cdot m^{-1} = l.$$

So while this satisfies P3, and is secure if Eve only sees encrypted messages, as soon as he knows what a message encodes to, then he can crack it.

One can replace e_k with any affine shift:

$$e_{(k_1,k_2)} : m \mapsto k_1 \cdot m + k_2.$$

Such so-called affine ciphers all fail the same *chosen plaintext attack*.

1.6.3 Back to encryption schemes

(This is after an interlude into complexity. See the complexity notes before this.)

In our public key encryption schemes, we have an encryption key e_k and a decryption key d_k. As a rule of thumb, the scheme is good if e_k and d_k are polynomial, but cryptoanalysis is exponential.

If encryption has complexity $f(k) = 10k^3 + 1000k$ while cryptoanalysis has complexity $g(k) = .2 \cdot 2^k$, then this isn't so good for blocksize $k = 10$, but choosing k properly, say $k = 60$ we get that $g(60) = 2 \times 10^{17}$ is about 10^{10} times as big as $f(60) \approx 2 \times 10^7$. Today's computers can do 10^7 flops in a second, it would take 100 years to do 10^{17}. So for the right blocksize, this is a good code.

Similarily, there can be good codes, according to our definition, that are good, but the blocksize necessary to ensure that cryptoanalysis is impossible also makes encryption impossible for today's computers. We still call them good though, because when computers get fast enough, they will become useful.

Problem 1.3. Explain this in a bit more detail.

1.2 Divisibility and GCDs

Recall that for integers a and b, we we say that a *divides* b, and write $a \mid b$, if there is another integer q, called the *quotient*, such that $aq = b$. If a divides b, then a is a *divisor* or *factor* of b, and b is a *multiple* of a.

Where all letters involved are integers it should be simple from this definition to prove such properties

i). $a|0$

ii). $0|a \Rightarrow a = 0$

iii). $1|a$

iv). $a|1 \Rightarrow a = \pm 1$

v). $a|b_1$ and $a|b_2 \Rightarrow a|c_1b_1 + c_2b_2$

vi). If $b + c = d$, and a divides two of b, c, d then it divides all of them.

With a bit more work you should be able to prove the the Division Algorithm: that for two integers a and b there is a unique pair of integers q and r with $0 \leq r < a$ such that

$$a = b \cdot q + r.$$

In this case r is called the *remainder* upon division of a by b.

The greatest common divisor $\gcd(a, b)$ of two integers a and b is the greatest integer that divides them both. From the last of the six properties given above, and the Division Algorithm, you should you know how to find the gcd of two integers using the *Euclidean Algorithm.*

Beginning with integers $a_1 \geq b_1$. At step i we set $a_i = b_{i-1}$, $b_i = r_{i-1}$ and divide a_i by b_i to get a remainder r_i. The algorithm stops when $r_i = 0$, and returns b_i as the GCD of a_0 and b_0.

Example 1.2. To find $\gcd(1253, 234)$ we use the Euclidean Algorithm:

Step	a	$=$	b	$\cdot$	q	$+$	r
1	1253	$=$	234	$\cdot$	5	$+$	83
2	234	$=$	83	$\cdot$	2	$+$	68
3	83	$=$	68	$\cdot$	1	$+$	15
4	68	$=$	15	$\cdot$	4	$+$	8
5	15	$=$	8	$\cdot$	1	$+$	7
6	8	$=$	7	$\cdot$	1	$+$	1
7	7	$=$	1	$\cdot$	7	$+$	0

So $\gcd(1253, 234) = 1$. Now to analyse the time this algorithm takes, there are two things that we consider. One is how many steps the algorithm requires, and the other is how long each step is.

In a typical step, we divide an integer a_i by a smaller integer b_i then multiply by q_i and subtract from a_i to get r_i. Perhaps one can do this more efficiently, but with this, the ith step has complexity $O((\log_2 a_i)^2) = O((\log_2 b_{i-1})^2)$.

Where k_i is the number of binary digits of b_i, without leading zeroes, step i has complexity $O(k_{i-1}^2)$.

Now the number of steps actually depends on b_0, we show the following.

Claim 1.3. *The algorithm takes at most* $2k_0 = 2\log_2(b_0)$ *steps.*

Proof. Considering r_0 as b_1 the algorithm stops after t steps, when $r_t = 0 \leq 1$. Observe that for $i \geq 0$ we have that $r_{i+2} < r_i/2$. Indeed $r_i = r_{i+1} \cdot q_i + r_{i+2}$, where $q_i \geq 1$ and $r_{i+1} > r_{i+2}$. So r_i is reduced by at least one binary digit every two steps. It is reduced to 1 binary digit in $2k_0$ steps. $\diamond$

So the GCD algorith takes $O(k_0)$ steps, having complexity (ignoring the first step) $O(k_0^2)$, then $O(k_1^2)$, then $O(k_2^2)$, etc. We know that $k_i \leq k_{i-1} - 1$, so where $k = k_0$ this is bounded in order by

$$2(k^2 + (k-1)^2 + \cdots + 1^2).$$

But we know that $\sum_{i=1}^{k} i^2 \approx k^3$, so the algorithm has complexity $O(k_0^3)$. Now we ignored that first step. This was order $O(k_a^2)$ where k_a is the number of binary digits of a. This could be important if a is a lot bigger than b. Some people might write that the algorithm is $O(k_a^2 + k_b^3)$. We assume that a and b are about the same size though, and say that GCD is cubic, having complexity $O(k^3)$.

As $\gcd(a, b)$ divides a and b, then it divides any linear combination $c_1 a + c_2 b$ of them for integers c_1 and c_2. Using this *Extended Euclidean Algorithm*, you should be able to write $\gcd(a, b)$ as a linear combination $\gcd(a, b) = c_1 a + c_2 b$ of a and b. So alternately we could define $\gcd(a, b)$ as the smallest positive linear combination of a and b with integer co-efficients.

Problem 1.4. Using the Extended Euclidean Algorithm (read Section 1.2 if you haven't seen it) express 1 as a linear combination of 1253 and 234.

Problem 1.5. Show that the Extended Euclidean Algorithm has complexity $O(k^3)$, (where we assume a and b both have about k binary digits).

1.3 Modular Arithmetic

You've probably seen Modular Arithmetic before, so we skip many details. The text fills in the details if you are unfamiliar.

Fixing an integer m, we say integers a and b are *congruent mod* m, and write $a \equiv b \mod m$ if $m \mid (a - b)$. Alternately, a and b are congruent mod m if and only if there exists some integer c such that $a = cm + b$.

Example 1.4.

$$-2 \equiv 5 \equiv 12 \equiv 712 \mod 7$$

Some basic properties of Modular Arithmetic are the following.

IF $a_1 \equiv b_1$ and $a_2 \equiv b_2 \mod m$ then

i). $a_1 a_2 \equiv b_1 b_2 \mod m$,

ii). $a_1 + a_2 \equiv b_1 + b_2 \mod m$,

iii). $a_1 - a_2 \equiv b_1 - b_2 \mod m$.

Problem 1.6. Prove these properties.

One sees that $a \equiv r \mod m$ where r is the remainder upon division of a by m:

$$a = q \cdot m + r$$

So every integer is congruent modulo m to some integer in $\{0, 1, \ldots, m-1\}$. In fact, reducing a to this remainder r is the canonical quotient map that takes the ring $\mathbb{Z}$ to the quotient ring $\mathbb{Z}/m\mathbb{Z} = \{0, 1, \ldots, m-1\}$ *modulo* the ideal $m\mathbb{Z}$.

The properties you proved in the above exersice are simply showing that the 'remainder' map is a homomorphism of $\mathbb{Z}$ to $\mathbb{Z}/m\mathbb{Z}$.

We say that b is a *multiplicative inverse* of a modulo m, if $ab \equiv 1 \mod m$. If a has a multiplicative inverse, we say it is *invertible modulo m.*

For example in $\mathbb{Z}/10\mathbb{Z}$, 7 has an inverse,

$$7 \cdot 3 = 21 \equiv 1 \mod 10,$$

but 5 does not, as all multiples of 5, $0, 5, 10, 15, 20, 25$ are congruent to 0 or 5 modulo 10.

Proposition 1.5. *Let $m \geq 1$ be an integer.*

i). If a is invertible modulo m, its inverse is unique modulo m, that is, if b and b' are inverses of a, then $b \equiv b' \mod m$.

ii). An integer a is invertible modulo m if and only if it is relatively prime to *m; that is, if and only if* $\gcd(a, m) = 1$.

Proof. The first property can be shown directly, or by observing that a has a multiplicative inverse modulo m if and only if the image of a in $\mathbb{Z}/m\mathbb{Z}$ is a unit.

For the second property, see Prop 1.13 of the text. □

In light of the first property, if a is invertible modulo m, we denote its inverse by a^{-1} or $1/a$. To calculate with it, we may replace it with any inverse of a modulo m. We tend to use the one in the range $1, \ldots, m-1$ and call this the inverse.

Example:

$$4/7 = 4 \cdot 7^{-1} \equiv 4 \cdot 3 = 12 \equiv 2 \mod 10.$$

Problem 1.7. Is 2536 invertible modulo 353? If so, what is it's inverse?

The set $(\mathbb{Z}/m\mathbb{Z})^*$ of units of $\mathbb{Z}/m\mathbb{Z}$ form a group. The *order* $|(\mathbb{Z}/m\mathbb{Z})^*|$ of this group of units will be important. It occurs enough that there is a function to describe it– *Euler's phi function:*

$$\phi(m) = ||(\mathbb{Z}/m\mathbb{Z})^*|.$$

Problem 1.8. What is $\phi(32)$?

In several applications, we will be computing large powers of an integer a in $\mathbb{Z}/m\mathbb{Z}$. To compute powers of integers a^b is exponential in a and b: it must be, even writing out a^b is exponential. Modulo m however, the size of a^b is essentially the same as the size of a. It is about m. If we compute 7^{415} and then reduce it modulo 3317, then we have to do 415 multiplications of increasingly large numbers. It seems clear that we will save time and space reducing the product at each step.

Problem 1.9. Compute 7^5 modulo 11.

But we still have to to b multiplications of two integers of size m. So the algorithm has $O(b)$ mutliplications of integers of size m. But this is still $O(2^{k_b}k_m^2)$, which is exponential in the number k_b of binary digits of b.

We can do better. Observe that $415 = 2^8 + 2^7 + 2^6 + 2^2 + 2 + 1$. So

$$7^{415} = 7^{2^8+2^7+2^6+2^2+2+1} = 7^{2^8} \cdot 7^{2^7} \cdot 7^{2^6} \cdot 7^{2^2} \cdot 7^2 \cdot 7.$$

It takes us $7 \approx \log_2(415)$ multiplications to evaluate all the factors 7^i modulo 3317, and about the same to multiply them together. This is 7 verses the 415 operations required to so it stupidly. This is called the fast powering algorithm.

Problem 1.10. What is the complexity of the fast powering algorithm?

Problem 1.11. Write a program in Sage to do fast powering. (Sage, of course, uses fast powering, but write it out yourself. The worksheet on the website does a slow powering algorithm.)

1.4 Prime numbers, Unique Factorisation, and finite fields

Recall that an integer $p \geq 2$ is *prime* if and only if for positive integers d

$$d \mid p \Rightarrow d = 1 \text{ or } p.$$

I expect that you've seen and could prove the following results:

- If a prime p divides ab for integers a and b, then it divides a or b.
- There are infinitely many primes.

The first of these is infact only true of 1 and primes, and their negatives. Further, you should have seen the following. You certainly believe it.

Theorem 1.6 (Fundamental Theorem of Arithmetic)**.** *Let $a \geq 2$ be an integer. Then a can be written as a product of primes:*

$$a = p_1^{e_1} \cdot p_2^{e_2} \cdot \ldots p_d^{e_d},$$

where each p_i is a distinct prime, and each e_i is a positive integer. Furthermore, except for order, this representation is unique.

This representation is called the *prime factorisation* of the integer a. For a particular prime p_i, the exponent e_i is called the *order of p_i in a*, and is denoted $e_i = \mathrm{ord}_{p_i}(a)$. If a prime p does not occur in in the prime factorisation of a, then $\mathrm{ord}_p(a) = 0$.

Example 1.7. The prime factorisation of 204 is

$$2^2 \cdot 3 \cdot 17,$$

so $\mathrm{ord}_2(204) = 2$, $\mathrm{ord}_{17}(204) = 1$, and $\mathrm{ord}_7(204) = 0$.

For prime p, the numbers $1, \ldots p-1$ are all relatively prime to p, and so we have the following useful fact.

Proposition 1.8. *If p is a prime then the group $(\mathbb{Z}/p\mathbb{Z})^*$ of units of $\mathbb{Z}/p\mathbb{Z}$ consists of the set*

$$\{1, 2, \ldots, p-1\}$$

of non-zero elements.

By Proposition 1.8, we have that for prime p, $\mathbb{Z}/p\mathbb{Z}$ is not just a ring, but a field. From field theory we know that the is a unique field of order p. This field is often denoted $\mathbb{F}_p$.

1.5 Powers and Primitive Roots in Finite Fields

We will use properties of $\mathbb{Z}/p\mathbb{Z} \cong \mathbb{F}_p$ that we know from field theory. Several of these properties, we can prove without much theory though, so we do that.

We have by Lagrange's theorem that ever element of a group has order dividing the order of the group: indeed the cosets of the subgroup generated by the element partition the group into equal parts. It follows that $a^{p-1} = 1$ for every element a in the multiplicitive group $(\mathbb{Z}/p\mathbb{Z})^*$, which has order $p-1$. This implies the following theorem, known as Fermat's Little Theorem; but we will prove it without applying Lagrange.

Theorem 1.9. *Let p be prime, and a be an integer. Then $a^{p-1} \equiv 0 \mod p$ if $p \mid a$ and otherwise $a^{p-1} \equiv 1 \mod p$.*

Proof. If $p|a$ then $a^{p-1} \equiv 0^{p-1} = 0$, so we may assume that $p \nmid a$. As the statement is about the image of a modulo p we may assume that a is its image in $(\mathbb{Z}/p\mathbb{Z})$. By the assumption $p \nmid a$, we then have that a is in $(\mathbb{Z}/p\mathbb{Z})^*$. The elements

$$a, 2a, \ldots (p-1)a$$

are distinct elements, for if $ia \equiv ja$ then $p \mid j - i$ which implies $i = j$. So we have

$$a^{p-1}(1 \cdot 2 \cdot \cdots \cdot p-1) = a \cdot 2a \cdot \cdots \cdot (p-1)a = 1 \cdot 2 \cdot \cdots \cdot (p-1).$$

Thus $a^{p-1} = 1$ in $(\mathbb{Z}/p\mathbb{Z})^*$, which means that $a^{p-1} \equiv 1 \mod p$. □

Immediate corollaries of this are the fact that for a with $\gcd(a, p) = 1$, we have

- $a^p \equiv a \mod p$
- $a^{-1} \equiv a^{p-2} \mod p$

This second seems nicer than the Extended Euclidean Algorithm for computing inverses modulo p. Conceptually it is easier, but computationally it is about the same: $2\log_2 p = O(k)$ multiplication/modular reductions of two k bit numbers, so $O(k^3)$.

The theorem also gives a nice check that a number is prime. Given an integer n the theorem gives us that if n is prime then for any smaller a, $a^{n-1} \equiv 1 \mod n$. So if we compute $2^{n-1} \mod n$ and get anything other than 1, then n is not prime. And we didn't have to find factors. It could be that we get a 1 when n is not prime, but doing the same calculation with a couple of different bases in place of 2, we can by pretty confident about whether or not n is prime. (However, this test is not perfect: there are known composite numbers n for which any a that is relatively prime to n yields $a^{n-1} \equiv 1 \mod n$. Such numbers are called Carmichael numbers.) We will look at better prime verification algorithms later.

Definition 1.10. For an integer a, the *order of a modulo p*, is its order in group $(\mathbb{Z}/p\mathbb{Z})^*$ that is, it is the minimum positive integer k such that $a^k \equiv 1 \mod p$.

Fermat's Little Theorem also gives us the following, (which you likely also know from group theory.)

Proposition 1.11. *Let p be a prime, and a be an integer not divisible by p. If $a^n \equiv 1 \mod p$ then the order of a modulo p divides n. In particular, it divides $p - 1$.*

Proof. See text. This is Prop 1.30. □

Now we know from field theory that any finite field has a cyclic multiplicative group. This is too much to prove here, even for finite fields $\mathbb{F}_p$ of prime order, (there are also finite fields of prime-power order), but we will use it.

Theorem 1.12 (The Primitive Root Theorem)**.** *Let p be a prime number. Then there exists an element $g \in \mathbb{F}_p^*$ such that*

$$\mathbb{F}_p^* = \{1, g, g^2, \ldots, g^{p-2}\}.$$

The elements g of the theorem are called *generators* or *primitive roots* of $\mathbb{F}_p^*$.

Problem 1.12. Find a primitive root of $\mathbb{F}_7^*$. How many are there?

You might do this by taking a element a to every power in $1, 2, \ldots, 6$, but you don't really have to do this. You only have to check certain powers. Which ones? In Excercise 1.30 from the text, you will have to do this for a prime p much bigger than 7, so think about this.

One can show that for k dividing $p-1$ there are exactly $\phi(k)$ elements in $\mathbb{F}_7^*$ having order k. You will do so in homework Question 1.31 of the text, only we don't know we are doing this yet. We will address properties of ϕ, later.

Notice that all the results of this section can easily be proved if we assume the Primitive Root Theorem.

Though $\mathbb{Z}/p\mathbb{Z}$ and $\mathbb{F}_p$ are isomorphic and the structure is clear when representing the field as $\mathbb{F}_p = \{0, 1, g, g^2, \ldots, g^{p-2}\}$ for some primitive root g, what we will often use is the fact that the structure in terms of $\mathbb{Z}/p\mathbb{Z}$, is not clear.

More concretely, we can fix an isomorphism $f : \mathbb{F}_p^* \to (\mathbb{Z}/p\mathbb{Z})^*$ by choosing a primitive root $f(g)$ in $(\mathbb{Z}/p\mathbb{Z})$. The sequence $f(g), f(g^2), f(g^3), \ldots, f(g^{p-1})$ list the elements of $(\mathbb{Z}/p\mathbb{Z})^*$ but mixes them up pretty good. (See Figure 2.2 from page 65 of the text.) One of the properties we will take advantage of is that there isn't any fast way to predict what integer x gives $f(g^x) = h$ for a number h in $(\mathbb{Z}/p\mathbb{Z})^*$.

Because of this we will use both $\mathbb{F}_p$ and $\mathbb{Z}/p\mathbb{Z}$. We will treat them notationally differently, but will exploit the fact that the are isomorphic.